

**hwk2/tb5\_byte\_word\_alu.vhd**

```

1  -----
2  --  TB5_ByteWordALU  --  SH-2 CPU Integration Testbench 5
3  --
4  --  Tests remaining byte/word data transfers, compares, EXTU.B, SUBC,
5  --  XOR #imm, TST #imm, MOVA, and remaining multi-bit shifts. PC relative offsets
6  --  will need to be adjusted when pipelining.
7  --
8  --  Instructions tested (26 checks):
9  --      MOV.W @(disp,PC),Rn      MOV.B Rm,@Rn + MOV.B @Rm,Rn
10 --      MOV.W Rm,@Rn + MOV.W @Rm,Rn
11 --      MOV.B @Rm+,Rn            MOV.W @Rm+,Rn
12 --      MOV.B Rm,@-Rn           MOV.W Rm,@-Rn
13 --      MOV.B R0,@(disp,Rn)     MOV.B @(disp,Rm),R0
14 --      MOV.W R0,@(disp,Rn)     MOV.W @(disp,Rm),R0
15 --      MOV.B Rm,@(R0,Rn)       MOV.B @(R0,Rm),Rn
16 --      MOV.W Rm,@(R0,Rn)       MOV.W @(R0,Rm),Rn
17 --      MOVA @(disp,PC),R0
18 --      CMP/EQ #imm,R0          CMP/GE Rm,Rn
19 --      CMP/HI Rm,Rn            CMP/PL Rn
20 --      CMP/STR Rm,Rn           EXTU.B Rm,Rn
21 --      SUBC Rm,Rn              XOR #imm,R0
22 --      TST #imm,R0             ROTR Rn
23 --      SHLL2 Rn      SHLL8 Rn      SHLR2 Rn      SHLR8 Rn
24 --
25 --  Disassembly (address : encoding : instruction):
26 --      0x0000: DE45  MOV.L @(0x45,PC),R14
27 --      0x0002: DF46  MOV.L @(0x46,PC),R15
28 --      0x0004: 0009  NOP
29 --      0x0006: 918B  MOV.W @(0x8B,PC),R1      R1 = 0x00001234
30 --      0x0008: 2E12  MOV.L R1,@R14
31 --      0x000A: 7E04  ADD #4,R14
32 --      0x000C: E142  MOV #0x42,R1
33 --      0x000E: 2E10  MOV.B R1,@R14      [R14] byte = 0x42
34 --      0x0010: 62E0  MOV.B @R14,R2      R2 = sign-ext 0x42 = 0x00000042
35 --      0x0012: 2E22  MOV.L R2,@R14      [R14] = 0x00000042
36 --      0x0014: 7E04  ADD #4,R14
37 --      0x0016: D142  MOV.L @(0x42,PC),R1
38 --      0x0018: 2E11  MOV.W R1,@R14      [R14] word = 0x1234
39 --      0x001A: 62E1  MOV.W @R14,R2      R2 = sign-ext 0x1234 = 0x00001234
40 --      0x001C: 2E22  MOV.L R2,@R14      [R14] = 0x00001234
41 --      0x001E: 7E04  ADD #4,R14
42 --      0x0020: E1AB  MOV #0xAB,R1      R1 = 0xFFFFFFFFAB (sign-ext)
43 --      0x0022: 2E10  MOV.B R1,@R14      [R14] byte = 0xAB
44 --      0x0024: 63E3  MOV R14,R3      R3 = R14 (save ptr)
45 --      0x0026: 6234  MOV.B @R3+,R2      R2 = sign-ext 0xAB = 0xFFFFFFFFAB
46 --      0x0028: 2E22  MOV.L R2,@R14      [R14] = 0xFFFFFFFFAB
47 --      0x002A: 7E04  ADD #4,R14
48 --      0x002C: D13D  MOV.L @(0x3D,PC),R1

```

```

49  -- 0x002E: 2E11 MOV.W R1,@R14      [R14] word = 0x8765
50  -- 0x0030: 63E3 MOV R14,R3
51  -- 0x0032: 6235 MOV.W @R3+,R2      R2 = sign-ext 0x8765 = 0xFFFF8765
52  -- 0x0034: 2E22 MOV.L R2,@R14      [R14] = 0xFFFF8765
53  -- 0x0036: 7E04 ADD #4,R14
54  -- 0x0038: E155 MOV #0x55,R1
55  -- 0x003A: 63E3 MOV R14,R3      R3 = R14
56  -- 0x003C: 7304 ADD #4,R3      R3 = R14+4
57  -- 0x003E: 2314 MOV.B R1,@-R3      [R3-1] byte = 0x55; R3 = R3-1
58  -- 0x0040: 6230 MOV.B @R3,R2      R2 = sign-ext 0x55 = 0x00000055
59  -- 0x0042: 2E22 MOV.L R2,@R14      [R14] = 0x00000055
60  -- 0x0044: 7E04 ADD #4,R14
61  -- 0x0046: D138 MOV.L @(0x38,PC),R1
62  -- 0x0048: 63E3 MOV R14,R3
63  -- 0x004A: 7304 ADD #4,R3
64  -- 0x004C: 2315 MOV.W R1,@-R3      [R3-2] word = 0x5678; R3 -= 2
65  -- 0x004E: 6231 MOV.W @R3,R2      R2 = sign-ext 0x5678 = 0x00005678
66  -- 0x0050: 2E22 MOV.L R2,@R14      [R14] = 0x00005678
67  -- 0x0052: 7E04 ADD #4,R14
68  -- 0x0054: E037 MOV #0x37,R0
69  -- 0x0056: 80E3 MOV.B R0,@(3,R14)  [R14+3] byte = 0x37
70  -- 0x0058: 84E3 MOV.B @(3,R14),R0  R0 = sign-ext 0x37 = 0x00000037
71  -- 0x005A: 2E02 MOV.L R0,@R14      [R14] = 0x00000037
72  -- 0x005C: 7E04 ADD #4,R14
73  -- 0x005E: D030 MOV.L @(0x30,PC),R0
74  -- 0x0060: 81E1 MOV.W R0,@(1*2,R14) [R14+2] word = 0x1234
75  -- 0x0062: 85E1 MOV.W @(1*2,R14),R0 R0 = sign-ext 0x1234 = 0x00001234
76  -- 0x0064: 2E02 MOV.L R0,@R14      [R14] = 0x00001234
77  -- 0x0066: 7E04 ADD #4,R14
78  -- 0x0068: E002 MOV #2,R0      R0 = 2 (offset)
79  -- 0x006A: E17E MOV #0x7E,R1
80  -- 0x006C: 0E14 MOV.B R1,@(R0,R14) [R14+2] byte = 0x7E
81  -- 0x006E: 02EC MOV.B @(R0,R14),R2 R2 = sign-ext 0x7E = 0x0000007E
82  -- 0x0070: 2E22 MOV.L R2,@R14      [R14] = 0x0000007E
83  -- 0x0072: 7E04 ADD #4,R14
84  -- 0x0074: E002 MOV #2,R0      R0 = 2
85  -- 0x0076: D12D MOV.L @(0x2D,PC),R1
86  -- 0x0078: 0E15 MOV.W R1,@(R0,R14) [R14+2] word = 0x2345
87  -- 0x007A: 02ED MOV.W @(R0,R14),R2 R2 = sign-ext 0x2345 = 0x00002345
88  -- 0x007C: 2E22 MOV.L R2,@R14      [R14] = 0x00002345
89  -- 0x007E: 7E04 ADD #4,R14
90  -- 0x0080: C72B MOVA @(0x2B,PC),R0  R0 = 0x0000012C
91  -- 0x0082: 2E02 MOV.L R0,@R14      store MOVA result
92  -- 0x0084: 7E04 ADD #4,R14
93  -- 0x0086: E02A MOV #42,R0
94  -- 0x0088: 882A CMP/EQ #42,R0      T = (42==42) = 1
95  -- 0x008A: 0129 MOVT R1
96  -- 0x008C: 2E12 MOV.L R1,@R14      [R14] = 1
97  -- 0x008E: 7E04 ADD #4,R14
98  -- 0x0090: E105 MOV #5,R1

```

```

99  --- 0x0092: E205 MOV #5,R2
100 --- 0x0094: 3123 CMP/GE R2,R1      T = (5>=5 signed) = 1
101 --- 0x0096: 0329 MOVT R3
102 --- 0x0098: 2E32 MOV.L R3,@R14     [R14] = 1
103 --- 0x009A: 7E04 ADD #4,R14
104 --- 0x009C: E10A MOV #10,R1
105 --- 0x009E: E205 MOV #5,R2
106 --- 0x00A0: 3126 CMP/HI R2,R1      T = (10>5 unsigned) = 1
107 --- 0x00A2: 0329 MOVT R3
108 --- 0x00A4: 2E32 MOV.L R3,@R14     [R14] = 1
109 --- 0x00A6: 7E04 ADD #4,R14
110 --- 0x00A8: E101 MOV #1,R1
111 --- 0x00AA: 4115 CMP/PL R1          T = (1>0) = 1
112 --- 0x00AC: 0329 MOVT R3
113 --- 0x00AE: 2E32 MOV.L R3,@R14     [R14] = 1
114 --- 0x00B0: 7E04 ADD #4,R14
115 --- 0x00B2: D11F MOV.L @(0x1F,PC),R1
116 --- 0x00B4: D21F MOV.L @(0x1F,PC),R2
117 --- 0x00B6: 212C CMP/STR R2,R1      T = 1 (byte 1 matches: 0x34)
118 --- 0x00B8: 0329 MOVT R3
119 --- 0x00BA: 2E32 MOV.L R3,@R14     [R14] = 1
120 --- 0x00BC: 7E04 ADD #4,R14
121 --- 0x00BE: E1FF MOV #0xFF,R1      R1 = 0xFFFFFFFF
122 --- 0x00C0: 621C EXTU.B R1,R2       R2 = 0x000000FF
123 --- 0x00C2: 2E22 MOV.L R2,@R14     [R14] = 0x000000FF
124 --- 0x00C4: 7E04 ADD #4,R14
125 --- 0x00C6: 0008 CLRT               T = 0
126 --- 0x00C8: E10A MOV #10,R1
127 --- 0x00CA: E203 MOV #3,R2
128 --- 0x00CC: 312A SUBC R2,R1          R1 = 10-3-T(0) = 7
129 --- 0x00CE: 2E12 MOV.L R1,@R14     [R14] = 0x00000007
130 --- 0x00D0: 7E04 ADD #4,R14
131 --- 0x00D2: E0FF MOV #0xFF,R0       R0 = 0xFFFFFFFF
132 --- 0x00D4: CA0F XOR #0x0F,R0       R0 = 0xFFFFFFFF0
133 --- 0x00D6: 2E02 MOV.L R0,@R14     [R14] = 0xFFFFFFFF0
134 --- 0x00D8: 7E04 ADD #4,R14
135 --- 0x00DA: E0F0 MOV #0xF0,R0       R0 = 0xFFFFFFFF0
136 --- 0x00DC: C80F TST #0x0F,R0       T = (0xFFFFFFFF0 & 0x0F == 0) = 1
137 --- 0x00DE: 0129 MOVT R1
138 --- 0x00E0: 2E12 MOV.L R1,@R14     [R14] = 1
139 --- 0x00E2: 7E04 ADD #4,R14
140 --- 0x00E4: D114 MOV.L @(0x14,PC),R1
141 --- 0x00E6: 4105 ROTR R1             R1 = 0xC0000000, T = 1
142 --- 0x00E8: 2E12 MOV.L R1,@R14     [R14] = 0xC0000000
143 --- 0x00EA: 7E04 ADD #4,R14
144 --- 0x00EC: E101 MOV #1,R1
145 --- 0x00EE: 4108 SHLL2 R1            R1 = 4
146 --- 0x00F0: 2E12 MOV.L R1,@R14     [R14] = 0x00000004
147 --- 0x00F2: 7E04 ADD #4,R14
148 --- 0x00F4: E101 MOV #1,R1

```

```

149  --      0x00F6: 4118  SHLL8 R1          R1 = 0x100
150  --      0x00F8: 2E12  MOV.L R1,@R14    [R14] = 0x00000100
151  --      0x00FA: 7E04  ADD #4,R14
152  --      0x00FC: E110  MOV #16,R1
153  --      0x00FE: 4109  SHLR2 R1          R1 = 4
154  --      0x0100: 2E12  MOV.L R1,@R14    [R14] = 0x00000004
155  --      0x0102: 7E04  ADD #4,R14
156  --      0x0104: D10D  MOV.L @(0x0D,PC),R1
157  --      0x0106: 4119  SHLR8 R1          R1 = 0xFF
158  --      0x0108: 2E12  MOV.L R1,@R14    [R14] = 0x000000FF
159  --      0x010A: 7E04  ADD #4,R14
160  --      0x010C: A000  BRA .              self-loop
161  --      0x010E: 0009  NOP              (delay slot)
162  --      0x0110: 0009  NOP
163  --      0x0112: 0009  NOP              (padding)
164  --      -- Data constants (at 0x0114) --
165  --      0x0114: 00001000
166  --      0x0118: 00001100
167  --      0x011C: 00001234
168  --      0x0120: FFFF8765
169  --      0x0124: 00005678
170  --      0x0128: 00002345
171  --      0x012C: 12345678
172  --      0x0130: AB34CDEF
173  --      0x0134: 80000001
174  --      0x0138: 0000FF00

```

175

176

177 **library** ieee;178 **use** ieee.std\_logic\_1164.all;179 **use** ieee.numeric\_std.all;180 **use** std.textio.all;

181

182 **entity** TB5\_ByteWordALU **is**183 **end entity**;

184

185 **architecture** sim **of** TB5\_ByteWordALU **is**

186

187     **constant** CLK\_PERIOD : **time**     **:=** 20 ns;188     **constant** PROG\_HEX   : **string**   **:=** "tb5\_program.hex";189     **constant** MEM\_WORDS  : **integer** **:=** 256;

190

191     **signal** clk        : **std\_logic** **:=** '0';192     **signal** reset     : **std\_logic** **:=** '0';193     **signal** loading   : **std\_logic** **:=** '1';

194

195     -- Shared bus

196     **signal** AB    : **std\_logic\_vector**(31 **downto** 0);197     **signal** DB    : **std\_logic\_vector**(31 **downto** 0);198     **signal** RE0, RE1, RE2, RE3 : **std\_logic**;

```

199     signal WE0, WE1, WE2, WE3 : std_logic;
200
201     -- TB-side bus signals
202     signal tb_AB : std_logic_vector(31 downto 0) := (others => '0');
203     signal tb_DB : std_logic_vector(31 downto 0) := (others => '0');
204     signal tb_RE : std_logic := '1';
205     signal tb_WE : std_logic := '1';
206
207 begin
208
209     clk <= not clk after CLK_PERIOD / 2;
210
211     AB <= tb_AB when loading = '1' else (others => 'Z');
212     DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
213     RE0 <= tb_RE when loading = '1' else 'Z';
214     RE1 <= tb_RE when loading = '1' else 'Z';
215     RE2 <= tb_RE when loading = '1' else 'Z';
216     RE3 <= tb_RE when loading = '1' else 'Z';
217     WE0 <= tb_WE when loading = '1' else 'Z';
218     WE1 <= tb_WE when loading = '1' else 'Z';
219     WE2 <= tb_WE when loading = '1' else 'Z';
220     WE3 <= tb_WE when loading = '1' else 'Z';
221
222     dut : entity work.SH2_CPU
223         port map (
224             Reset => reset,  NMI => '1',  INT => '1',  clock => clk,
225             AB  => AB,    DB  => DB,
226             RE0 => RE0,  RE1 => RE1,  RE2 => RE2,  RE3 => RE3,
227             WE0 => WE0,  WE1 => WE1,  WE2 => WE2,  WE3 => WE3
228         );
229
230     mem : entity work.MEMORY32x32
231         generic map (
232             MEMSIZE      => MEM_WORDS,
233             START_ADDR0  => 16#00000000#,
234             START_ADDR1  => 16#00001000#,
235             START_ADDR2  => 16#10000000#,
236             START_ADDR3  => 16#20000000#
237         )
238         port map (
239             RE0 => RE0,  RE1 => RE1,  RE2 => RE2,  RE3 => RE3,
240             WE0 => WE0,  WE1 => WE1,  WE2 => WE2,  WE3 => WE3,
241             MemAB => AB,  MemDB => DB
242         );
243
244     stim : process
245         file      hex_file : text;
246         variable fstatus   : file_open_status;
247         variable l          : line;
248         variable word       : std_logic_vector(31 downto 0);

```

```

249     variable addr      : unsigned(31 downto 0);
250     variable pass_cnt  : integer := 0;
251     variable fail_cnt  : integer := 0;
252
253     procedure check(a   : std_logic_vector(31 downto 0);
254                    exp  : std_logic_vector(31 downto 0);
255                    tag  : string) is
256     begin
257         tb_AB <= a;
258         tb_RE <= '0';
259         tb_WE <= '1';
260         wait for CLK_PERIOD;
261         if DB = exp then
262             pass_cnt := pass_cnt + 1;
263             report "PASS: " & tag severity note;
264         else
265             fail_cnt := fail_cnt + 1;
266             report "FAIL: " & tag
267                 & " got=0x" & to_hstring(DB)
268                 & " exp=0x" & to_hstring(exp) severity error;
269         end if;
270         tb_RE <= '1';
271         wait for CLK_PERIOD;
272     end procedure;
273
274 begin
275     -----
276     -- LOAD PHASE
277     -----
278     loading <= '1';
279     reset   <= '0';
280     tb_RE   <= '1';
281     tb_WE   <= '1';
282     wait for CLK_PERIOD * 2;
283
284     file_open(fstatus, hex_file, PROG_HEX, read_mode);
285     assert fstatus = open_ok
286         report "Cannot open " & PROG_HEX severity failure;
287
288     addr := (others => '0');
289     while not endfile(hex_file) loop
290         readline(hex_file, l);
291         if l'length > 0 then
292             hread(l, word);
293             tb_AB <= std_logic_vector(addr);
294             tb_DB <= word;
295             tb_WE <= '0';
296             wait for CLK_PERIOD;
297             tb_WE <= '1';
298             wait for CLK_PERIOD;

```

```

299         addr := addr + 4;
300     end if;
301 end loop;
302 file_close(hex_file);
303
304 -----
305 -- CPU PHASE
306 -----
307 wait until rising_edge(clk);
308 loading <= '0';
309 reset    <= '1';
310
311 wait for CLK_PERIOD * 2;
312
313 for i in 1 to 400 loop
314     wait until rising_edge(clk);
315 end loop;
316
317 -----
318 -- CHECK PHASE
319 -----
320 wait until rising_edge(clk);
321 reset    <= '0';
322 loading <= '1';
323 tb_RE    <= '1';
324 tb_WE    <= '1';
325 wait for CLK_PERIOD * 2;
326
327 check(x"00001000", x"00001234", "MOV.W @(disp,PC)");
328 check(x"00001004", x"00000042", "MOV.B Rm,@Rn + MOV.B @Rm,Rn");
329 check(x"00001008", x"00001234", "MOV.W Rm,@Rn + MOV.W @Rm,Rn");
330 check(x"0000100C", x"FFFFFFAB", "MOV.B @Rm+,Rn");
331 check(x"00001010", x"FFFF8765", "MOV.W @Rm+,Rn");
332 check(x"00001014", x"00000055", "MOV.B Rm,@-Rn");
333 check(x"00001018", x"00005678", "MOV.W Rm,@-Rn");
334 check(x"0000101C", x"00000037", "MOV.B R0,@(disp,Rn) + MOV.B @(disp,Rm),R0");
335 check(x"00001020", x"00001234", "MOV.W R0,@(disp,Rn) + MOV.W @(disp,Rm),R0");
336 check(x"00001024", x"0000007E", "MOV.B Rm,@(R0,Rn) + MOV.B @(R0,Rm),Rn");
337 check(x"00001028", x"00002345", "MOV.W Rm,@(R0,Rn) + MOV.W @(R0,Rm),Rn");
338 check(x"0000102C", x"0000012C", "MOVA @(disp,PC),R0");
339 check(x"00001030", x"00000001", "CMP/EQ #imm,R0");
340 check(x"00001034", x"00000001", "CMP/GE Rm,Rn");
341 check(x"00001038", x"00000001", "CMP/HI Rm,Rn");
342 check(x"0000103C", x"00000001", "CMP/PL Rn");
343 check(x"00001040", x"00000001", "CMP/STR Rm,Rn");
344 check(x"00001044", x"000000FF", "EXTU.B Rm,Rn");
345 check(x"00001048", x"00000007", "SUBC Rm,Rn");
346 check(x"0000104C", x"FFFFFFF0", "XOR #imm,R0");
347 check(x"00001050", x"00000001", "TST #imm,R0");
348 check(x"00001054", x"C0000000", "ROTR Rn");

```

```
349     check(x"00001058", x"00000004", "SHLL2 Rn");
350     check(x"0000105C", x"00000100", "SHLL8 Rn");
351     check(x"00001060", x"00000004", "SHLR2 Rn");
352     check(x"00001064", x"000000FF", "SHLR8 Rn");
353
354     -----
355     -- SUMMARY
356     -----
357     report "TB5: PASSED=" & integer'image(pass_cnt)
358         & "   FAILED=" & integer'image(fail_cnt) severity note;
359     if fail_cnt = 0 then
360         report "*** ALL TESTS PASSED ***" severity note;
361     else
362         assert false
363             report integer'image(fail_cnt) & " TEST(S) FAILED"
364             severity failure;
365     end if;
366     wait;
367 end process;
368
369 end architecture;
370
```